

LAPORAN PRAKTIKUM

STRUKTUR DATA

“SORTING”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHARAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “SORTING” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya materi Sorting pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II	3
2.1 Langkah Kerja Praktikum	3
2.2 Analisis Hasil dan Pembahasan	19
BAB III.....	22
3.1 Kesimpulan	22
DAFTAR PUSTAKA	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, sering kali kita menemukan data yang tersusun secara acak sehingga sulit untuk dibaca maupun diproses. Oleh karena itu, diperlukan suatu metode untuk mengurutkan data agar lebih terstruktur dan mudah digunakan. Proses pengurutan data ini dikenal dengan istilah *sorting*.

Pada praktikum kali ini, dipelajari beberapa algoritma sorting yaitu Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort. Setiap algoritma memiliki cara kerja yang berbeda dalam mengurutkan data. Bubble Sort melakukan pengurutan dengan membandingkan elemen yang berdekatan, Merge Sort menggunakan metode pembagian dan penggabungan data, Quick Sort memanfaatkan pivot untuk membagi data, sedangkan Shell Sort menggunakan jarak tertentu (*gap*) untuk mempercepat proses pengurutan.

Melalui praktikum ini, mahasiswa dapat memahami cara kerja masing-masing algoritma sorting serta mengetahui perbedaan proses pengurutan yang dilakukan oleh setiap metode. Selain itu, praktikum ini juga membantu mahasiswa meningkatkan kemampuan dalam mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar pengurutan data (*sorting*) dalam struktur data.
2. Mempelajari cara kerja algoritma Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort.
3. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
4. Mengetahui proses pengurutan data secara bertahap melalui tampilan program GUI.
5. Membandingkan cara kerja beberapa algoritma sorting dalam menyelesaikan proses pengurutan data.

1.3 Manfaat Praktikum

Manfaat yang didapat dari praktikum ini adalah:

1. Menambah pemahaman mengenai konsep sorting dalam struktur data.
2. Melatih kemampuan pemrograman Java dalam mengimplementasikan algoritma pengurutan.
3. Membantu memahami langkah-langkah pengurutan data secara visual melalui program GUI.
4. Mengetahui karakteristik dan perbedaan dari setiap algoritma sorting yang digunakan.
5. Menjadi dasar dalam mempelajari algoritma dan struktur data yang lebih kompleks pada perkuliahan selanjutnya.

BAB II PEMBAHASAN

2.1 Langkah Kerja Praktikum

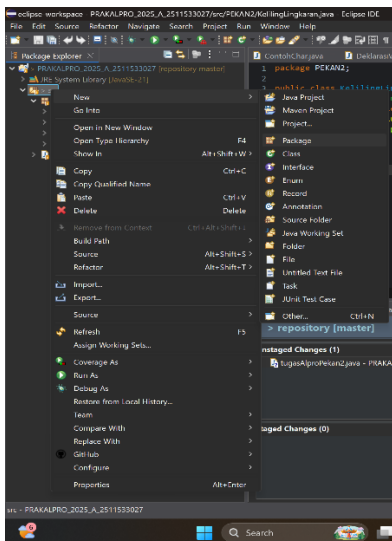
1. Persiapan Awal

- Buka apl Eclipse IDE di laptop atau di komputer.

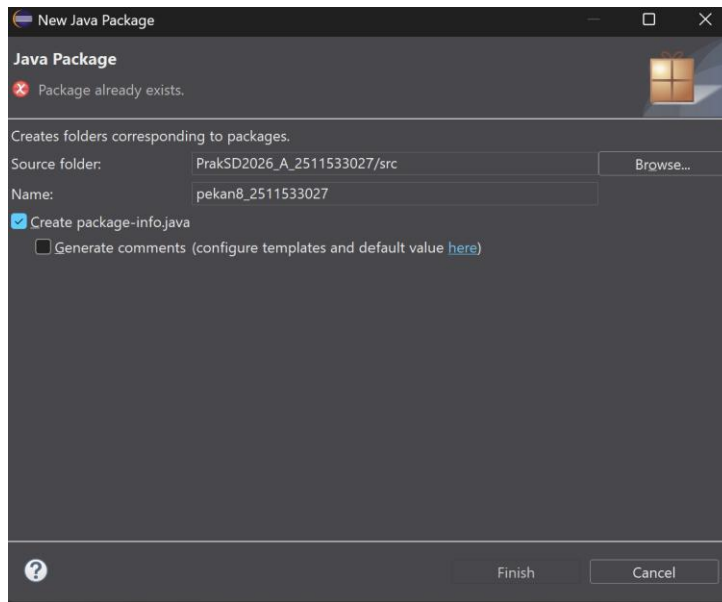


2. Membuat Package

- Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

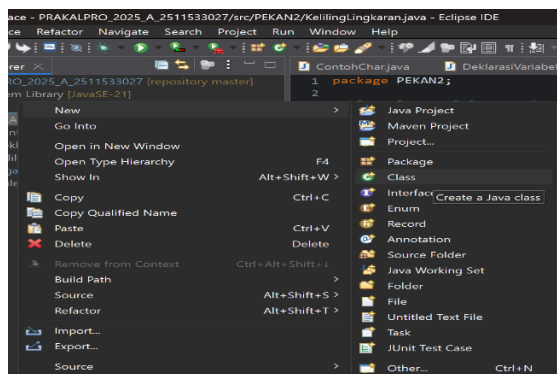


- Beri nama package **pekan8_NIM**.



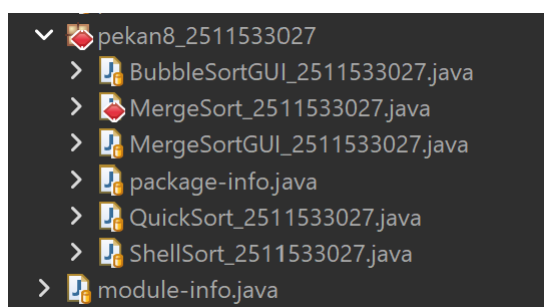
3. Membuat Class Baru untuk Setiap Percobaan

a) Klik kanan pada package **pekan8_NIM**, lalu pilih **New**, terakhir klik **Class**.



1. Buat beberapa kelas sesuai percobaan.

- BubbleSortGUI_2511533027.java
- MergeSort_2511533027.java
- MergeSortGUI_2511533027.java
- QuickSort_2511533027.java
- ShellSort_2511533027.java



4. Menulis Kode Program

1. BubbleSortGUI_2511533027.java

- Program ini digunakan untuk memahami proses pengurutan data menggunakan algoritma Bubble Sort secara visual melalui Graphical User Interface (GUI).
- Program menerima input berupa beberapa angka yang dipisahkan dengan tanda koma.
- Method `setArrayFromInput_3027()` digunakan untuk membaca dan menyimpan data yang dimasukkan pengguna ke dalam array.
- Method `performStep_3027()` digunakan untuk menjalankan proses Bubble Sort secara bertahap dengan membandingkan dua elemen yang bersebelahan.
- Program memberikan warna pada elemen yang sedang dibandingkan dan elemen yang ditukar sehingga proses sorting lebih mudah dipahami.
- Setiap langkah pengurutan ditampilkan pada area log untuk menunjukkan perubahan array yang terjadi.
- Hasil akhir program berupa array yang telah terurut secara ascending serta tampilan visual dari proses Bubble Sort.

```
package pekan8_2511533027;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class BubbleSortGUI_2511533027 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_3027;
    private JLabel[] labelArray_3027;
    private JButton stepButton_3027, resetButton_3027, setButton_3027;
    private JTextField inputField_3027;
    private JPanel panelArray_3027;
    private JTextArea stepArea_3027;
```



```

private int i_3027 = 1, j_3027;
private boolean sorting_3027 = false;
private int stepCount_3027 = 1;

/**
 * Launch the application.
 */

/**
 * Create the frame.
 */
/**
 * Create the frame.
 */
public BubbleSortGUI_2511533027() {
    setTitle("Bubble Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // Panel input
    JPanel inputPanel_3027 = new JPanel(new FlowLayout());
    inputField_3027 = new JTextField(30);
    setButton_3027 = new JButton("Set Array");
    inputPanel_3027.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
    inputPanel_3027.add(inputField_3027);
    inputPanel_3027.add(setButton_3027);

    // Panel Array visual
    panelArray_3027 = new JPanel();
    panelArray_3027.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel_3027 = new JPanel();
    stepButton_3027 = new JButton("Langkah Selanjutnya");
    resetButton_3027 = new JButton("Reset");
    stepButton_3027.setEnabled(false);
    controlPanel_3027.add(stepButton_3027);
    controlPanel_3027.add(resetButton_3027);

    // Area text untuk log langkah-langkah
    stepArea_3027 = new JTextArea(8, 60);
    stepArea_3027.setEditable(false);
    stepArea_3027.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane_3027 = new JScrollPane(stepArea_3027);

    // Tambahkan panel ke frame
    add(inputPanel_3027, BorderLayout.NORTH);
    add(panelArray_3027, BorderLayout.CENTER);
    add(controlPanel_3027, BorderLayout.SOUTH);
    add(scrollPane_3027, BorderLayout.EAST);

    // Event Set Array
    setButton_3027.addActionListener(e ->
setArrayFromInput_3027());

    // Event Langkah Selanjutnya

```

```

        stepButton_3027.addActionListener(e -> performStep_3027());

        // Event Reset
        resetButton_3027.addActionListener(e -> reset_3027());
    }

    private void setArrayFromInput_3027() {
        String text_3027 = inputField_3027.getText().trim();
        if (text_3027.isEmpty()) return;
        String[] parts_3027 = text_3027.split(",");
        array_3027 = new int[parts_3027.length];
        try {
            for (int k_3027 = 0; k_3027 < parts_3027.length;
k_3027++) {
                array_3027[k_3027] =
Integer.parseInt(parts_3027[k_3027].trim());
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(this, "Masukkan
hanya angka "
                    + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
                return;
            }
            i_3027 = 0;
            j_3027 = 0;
            stepCount_3027 = 1;
            sorting_3027 = true;
            stepButton_3027.setEnabled(true);
            stepArea_3027.setText("");
            panelArray_3027.removeAll();
            labelArray_3027 = new JLabel[array_3027.length];
            for (int k_3027 = 0; k_3027 < array_3027.length;
k_3027++) {
                labelArray_3027[k_3027] = new
JLabel(String.valueOf(array_3027[k_3027]));
                labelArray_3027[k_3027].setFont(new Font("Arial",
Font.BOLD, 24));
                labelArray_3027[k_3027].setOpaque(true);

                labelArray_3027[k_3027].setBackground(Color.WHITE);

                labelArray_3027[k_3027].setBorder(BorderFactory.createLineBorder(Col
or.BLACK));
                labelArray_3027[k_3027].setPreferredSize(new
Dimension(50, 50));

                labelArray_3027[k_3027].setHorizontalAlignment(SwingConstants.CENTER
);
                panelArray_3027.add(labelArray_3027[k_3027]);
            }

            panelArray_3027.revalidate();
            panelArray_3027.repaint();
        }

        private void performStep_3027() {
            if (!sorting_3027 || i_3027 >= array_3027.length - 1) {
                sorting_3027 = false;
                stepButton_3027.setEnabled(false);
                JOptionPane.showMessageDialog(this, "Sorting selesai!");
            }
        }
    }

```

```

        return;
    }
    resetHighlights_3027();
    StringBuilder stepLog_3027 = new StringBuilder();
    labelArray_3027[j_3027].setBackground(Color.CYAN);
    labelArray_3027[j_3027 + 1].setBackground(Color.CYAN);
    if (array_3027[j_3027] > array_3027[j_3027 + 1]) {
        // Swap
        int temp_3027 = array_3027[j_3027];
        array_3027[j_3027] = array_3027[j_3027 + 1];
        array_3027[j_3027 + 1] = temp_3027;
        labelArray_3027[j_3027].setBackground(Color.RED);
        labelArray_3027[j_3027 + 1].setBackground(Color.RED);
        stepLog_3027.append("Langkah
    ").append(stepCount_3027).append(": Menukar elemen ke-")
            .append(j_3027).append("
    ").append(array_3027[j_3027 + 1]).append(" dengan ke-")
            .append(j_3027 + 1).append("
    ").append(array_3027[j_3027]).append(")\n");
    } else {
        stepLog_3027.append("Langkah
    ").append(stepCount_3027).append(": Tidak ada pertukaran antara ke-")
            .append(j_3027).append(" dan ke-
    ").append(j_3027 + 1).append(")\n");
    }
    stepLog_3027.append("Hasil:
    ").append(arrayToString_3027(array_3027)).append("\n\n");
    stepArea_3027.append(stepLog_3027.toString());
    updateLabels_3027();
    j_3027++;
    if (j_3027 >= array_3027.length - i_3027 - 1) {
        j_3027 = 0;
        i_3027++;
    }
    stepCount_3027++;
    if (i_3027 >= array_3027.length - 1) {
        sorting_3027 = false;
        stepButton_3027.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void updateLabels_3027() {
    for (int k_3027 = 0; k_3027 < array_3027.length; k_3027++) {
labelArray_3027[k_3027].setText(String.valueOf(array_3027[k_3027]));
    }
}

private void resetHighlights_3027() {
    for (JLabel label : labelArray_3027) {
        label.setBackground(Color.WHITE);
    }
}

private void reset_3027() {
    inputField_3027.setText("");
    panelArray_3027.removeAll();
    panelArray_3027.revalidate();
    panelArray_3027.repaint();
    stepArea_3027.setText("");
    stepButton_3027.setEnabled(false);
}

```

```

        sorting_3027 = false;
        i_3027 = 0;
        j_3027 = 0;
        stepCount_3027 = 1;
    }

    private String arrayToString_3027(int[] arr) {
        StringBuilder sb = new StringBuilder();
        for (int k = 0; k < arr.length; k++) {
            sb.append(arr[k]);
            if (k < arr.length - 1) sb.append(", ");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BubbleSortGUI_2511533027 gui_3027 = new
BubbleSortGUI_2511533027();
            gui_3027.setVisible(true);
        });
    }
}

```

2. MergeSort_2511533027.java

- Program ini digunakan untuk memahami proses pengurutan data menggunakan algoritma Merge Sort.
- Program bekerja dengan cara membagi array menjadi beberapa bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut.
- Method sort() digunakan untuk membagi array secara rekursif menjadi dua bagian hingga tersisa satu elemen.
- Method merge_3027() digunakan untuk menggabungkan dua subarray yang telah terurut menjadi satu array yang lebih besar.
- Program menggunakan array sementara untuk membantu proses penggabungan data.
- Pada method main, program mendeklarasikan array integer yang belum terurut dan menjalankan proses Merge Sort.
- Hasil akhir program berupa array yang telah terurut menggunakan algoritma Merge Sort.

```

package pekan8_2511533027;

public class MergeSort_2511533027 {
    void merge_3027(int arr_3027[], int l_3027, int m_3027, int
r_3027) {
        /* Find sizes of two subarrays to be merged */
        int n1_3027 = m_3027 - l_3027 + 1;

```

```

int n2_3027 = r_3027 - m_3027;

/* Create temp arrays */
int L_3027[] = new int[n1_3027];
int R_3027[] = new int[n2_3027];

/* Copy data to temp arrays */
for (int i_3027 = 0; i_3027 < n1_3027; ++i_3027)
    L_3027[i_3027] = arr_3027[l_3027 + i_3027];
for (int j_3027 = 0; j_3027 < n2_3027; ++j_3027)
    R_3027[j_3027] = arr_3027[m_3027 + 1 + j_3027];

int i_3027 = 0, j_3027 = 0;
int k_3027 = l_3027;
while (i_3027 < n1_3027 && j_3027 < n2_3027) {
    if (L_3027[i_3027] <= R_3027[j_3027]) {
        arr_3027[k_3027] = L_3027[i_3027];
        i_3027++;
    } else {
        arr_3027[k_3027] = R_3027[j_3027];
        j_3027++;
    }
    k_3027++;
}
while (i_3027 < n1_3027) {
    arr_3027[k_3027] = L_3027[i_3027];
    i_3027++;
    k_3027++;
}

while (j_3027 < n2_3027) {
    arr_3027[k_3027] = R_3027[j_3027];
    j_3027++;
    k_3027++;
}
}

void sort(int arr_3027[], int l_3027, int r_3027) {
    if (l_3027 < r_3027) {
        // Find the middle point
        int m_3027 = (l_3027 + r_3027) / 2;

        // Sort first and second halves
        sort(arr_3027, l_3027, m_3027);
        sort(arr_3027, m_3027 + 1, r_3027);

        // Merge the sorted halves
        merge_3027(arr_3027, l_3027, m_3027, r_3027);
    }
}

/* A utility function to print array of size n */
static void printArray(int arr_3027[]) {
    int n_3027 = arr_3027.length;
    for (int i_3027 = 0; i_3027 < n_3027; i_3027++)
        System.out.print(arr_3027[i_3027] + " ");
    System.out.println();
}

```

```

        public static void main(String args_3027[]) {
            int arr_3027[] = { 12, 11, 13, 5, 6, 7 };
            System.out.println("Sebelum terurut");
            printArray(arr_3027);

            MergeSort_2511533027 ob_3027 = new MergeSort_2511533027();
            ob_3027.sort(arr_3027, 0, arr_3027.length - 1);
            System.out.println("\nSesudah Terurut menggunakan merge
Sort");
            printArray(arr_3027);
        }
    }
}

```

3. MergeSortGUI_2511533027.java

- Program ini digunakan untuk memahami proses pengurutan data menggunakan algoritma Merge Sort secara visual melalui GUI.
- Program menerima input berupa sekumpulan angka yang dipisahkan dengan tanda koma.
- Method generateMergeSteps_3027() digunakan untuk menentukan tahapan penggabungan yang akan dilakukan selama proses Merge Sort.
- Method performStep_3027() digunakan untuk menjalankan setiap langkah penggabungan secara bertahap.
- Program menampilkan elemen yang sedang dibandingkan, disalin, dan ditempatkan kembali ke array utama menggunakan warna yang berbeda.
- Setiap langkah proses Merge Sort dicatat pada area log sehingga pengguna dapat mengikuti proses pengurutan dengan lebih mudah.
- Hasil akhir program berupa array yang telah terurut serta visualisasi proses Merge Sort langkah demi langkah.

```

package pekan8_2511533027;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;

```

```

import javax.swing.SwingUtilities;

import java.util.LinkedList;
import java.util.Queue;

public class MergeSortGUI_2511533027 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_3027;
    private JLabel[] labelArray_3027;
    private JButton stepButton_3027, resetButton_3027, setButton_3027;
    private JTextField inputField_3027;
    private JPanel panelArray_3027;
    private JTextArea stepArea_3027;

    private Queue<int[]> mergeQueue_3027 = new LinkedList<>();
    private int stepCount_3027 = 1;
    private boolean isMerging_3027 = false;
    private boolean copying_3027 = false;

    private int[] temp_3027;
    private int left_3027, mid_3027, right_3027;
    private int i_3027, j_3027, k_3027;

    public MergeSortGUI_2511533027() {
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // panel input
        JPanel inputPanel_3027 = new JPanel(new FlowLayout());
        inputField_3027 = new JTextField(30);
        setButton_3027 = new JButton("Set Array");
        inputPanel_3027.add(new JLabel("Masukan angka (pisahkan dengan
koma)"));
        inputPanel_3027.add(inputField_3027);
        inputPanel_3027.add(setButton_3027);

        // panel array visual
        panelArray_3027 = new JPanel();
        panelArray_3027.setLayout(new FlowLayout());

        // panel kontrol
        JPanel controlPanel_3027 = new JPanel();
        stepButton_3027 = new JButton("Langkah selanjutnya");
        resetButton_3027 = new JButton("Reset");
        controlPanel_3027.add(stepButton_3027);
        controlPanel_3027.add(resetButton_3027);

        // area text untuk log langkah langkah
        stepArea_3027 = new JTextArea(8, 60);
        stepArea_3027.setEditable(false);
        stepArea_3027.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane_3027 = new JScrollPane(stepArea_3027);

        // tambahkan panel ke frame
        add(inputPanel_3027, BorderLayout.NORTH);
    }
}

```

```

        add(panelArray_3027, BorderLayout.CENTER);
        add(controlPanel_3027, BorderLayout.SOUTH);
        add(scrollPane_3027, BorderLayout.EAST);

        // event set array
        setButton_3027.addActionListener(e ->
setArrayFromInput_3027());

        // event selanjutnya
        stepButton_3027.addActionListener(e -> performStep_3027());

        // event reset
        resetButton_3027.addActionListener(e -> reset_3027());
    }

    private void setArrayFromInput_3027() {
        String text_3027 = inputField_3027.getText().trim();
        if (text_3027.isEmpty()) return;
        String[] parts_3027 = text_3027.split(",");
        array_3027 = new int[parts_3027.length];
        try {
            for (int k = 0; k < parts_3027.length; k++) {
                array_3027[k] =
Integer.parseInt(parts_3027[k].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka dengan koma!",
                "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        labelArray_3027 = new JLabel[array_3027.length];
        panelArray_3027.removeAll();
        for (int k = 0; k < array_3027.length; k++) {
            labelArray_3027[k] = new
JLabel(String.valueOf(array_3027[k]));
            labelArray_3027[k].setFont(new Font("Arial", Font.BOLD,
24));

            labelArray_3027[k].setOpaque(true);
            labelArray_3027[k].setBackground(Color.WHITE);

            labelArray_3027[k].setBorder(BorderFactory.createLineBorder(Color.BL
ACK));

            labelArray_3027[k].setPreferredSize(new Dimension(50,
50));

            labelArray_3027[k].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_3027.add(labelArray_3027[k]);
        }

        mergeQueue_3027.clear();
        generateMergeSteps_3027(0, array_3027.length - 1);
        stepButton_3027.setEnabled(true);
        stepArea_3027.setText("");
        stepCount_3027 = 1;
        isMerging_3027 = false;

        panelArray_3027.revalidate();
    }

```



```

        panelArray_3027.repaint();
    }

    private void generateMergeSteps_3027(int left, int right) {
        if (left >= right) return;
        int mid = (left + right) / 2;
        generateMergeSteps_3027(left, mid);
        generateMergeSteps_3027(mid + 1, right);
        mergeQueue_3027.add(new int[]{left, mid, right});
    }

    private void performStep_3027() {
        resetHighlights_3027();

        if (!isMerging_3027 && !mergeQueue_3027.isEmpty()) {
            int[] range_3027 = mergeQueue_3027.poll();
            left_3027 = range_3027[0];
            mid_3027 = range_3027[1];
            right_3027 = range_3027[2];
            temp_3027 = new int[right_3027 - left_3027 + 1];
            i_3027 = left_3027;
            j_3027 = mid_3027 + 1;
            k_3027 = 0;
            copying_3027 = false;
            isMerging_3027 = true;
            stepArea_3027.append("Langkah " + stepCount_3027++ +
                                ": Mulai merge dari " + left_3027 + " ke "
+ right_3027 + "\n");
            return;
        }

        if (isMerging_3027 && !copying_3027) {
            if (i_3027 <= mid_3027 && j_3027 <= right_3027) {

                labelArray_3027[i_3027].setBackground(Color.CYAN);

                labelArray_3027[j_3027].setBackground(Color.CYAN);
                if (array_3027[i_3027] <= array_3027[j_3027]) {
                    temp_3027[k_3027++] =
array_3027[i_3027++];
                } else {
                    temp_3027[k_3027++] =
array_3027[j_3027++];
                }
                stepArea_3027.append("Langkah " +
stepCount_3027++ + ": Bandingkan dan salin elemen\n");
                return;
            } else if (i_3027 <= mid_3027) {
                temp_3027[k_3027++] = array_3027[i_3027++];
                stepArea_3027.append("Langkah " +
stepCount_3027++ + ": Salin sisa kiri\n");
                return;
            } else if (j_3027 <= right_3027) {
                temp_3027[k_3027++] = array_3027[j_3027++];
                stepArea_3027.append("Langkah " +
stepCount_3027++ + ": Salin sisa kanan\n");
                return;
            } else {
                copying_3027 = true;

```

```

        k_3027 = 0;
        return;
    }

    if (copying_3027 && k_3027 < temp_3027.length) {
        array_3027[left_3027 + k_3027] = temp_3027[k_3027];
        labelArray_3027[left_3027 +
k_3027].setText(String.valueOf(temp_3027[k_3027]));
        labelArray_3027[left_3027 +
k_3027].setBackground(Color.GREEN);
        k_3027++;
        stepArea_3027.append("Langkah " + stepCount_3027++ + ":
Tempelkan ke array utama\n");
        return;
    }

    if (copying_3027 && k_3027 == temp_3027.length) {
        isMerging_3027 = false;
        copying_3027 = false;
    }

    if (mergeQueue_3027.isEmpty() && !isMerging_3027) {
        stepArea_3027.append("Selesai.\n");
        stepButton_3027.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort
selesai!");
    }
}

private void resetHighlights_3027() {
    if (labelArray_3027 == null) return;
    for (JLabel label : labelArray_3027) {
        label.setBackground(Color.WHITE);
    }
}

private void reset_3027() {
    inputField_3027.setText("");
    panelArray_3027.removeAll();
    panelArray_3027.revalidate();
    panelArray_3027.repaint();
    stepArea_3027.setText("");
    stepButton_3027.setEnabled(false);
    mergeQueue_3027.clear();
    isMerging_3027 = false;
    stepCount_3027 = 1;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGUI_2511533027 gui = new
MergeSortGUI_2511533027();
        gui.setVisible(true);
    });
}
}

```

4. QuickSort_2511533027.java

- Program ini digunakan untuk memahami proses pengurutan data menggunakan algoritma Quick Sort.
- Program bekerja dengan memilih sebuah pivot dan membagi data menjadi dua bagian berdasarkan nilai pivot tersebut.
- Method `medianOfThree_3027()` digunakan untuk menentukan pivot menggunakan teknik Median of Three agar proses pengurutan lebih optimal.
- Method `partition_3027()` digunakan untuk menempatkan pivot pada posisi yang tepat dan mengatur elemen yang lebih kecil atau lebih besar dari pivot.
- Method `quickSort_3027()` digunakan untuk menjalankan proses pengurutan secara rekursif pada bagian kiri dan kanan pivot.
- Pada method `main`, program mendeklarasikan array integer yang belum terurut dan menjalankan algoritma Quick Sort.
- Hasil akhir program berupa array yang telah terurut secara ascending menggunakan algoritma Quick Sort.

```
package pekan8_2511533027;

public class QuickSort_2511533027 {
    static void swap_3027(int[] arr_3027, int i_3027, int j_3027) {
        int temp_3027 = arr_3027[i_3027];
        arr_3027[i_3027] = arr_3027[j_3027];
        arr_3027[j_3027] = temp_3027;
    }

    // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_3027(int[] arr_3027, int low_3027, int
high_3027) {
        int mid_3027 = low_3027 + (high_3027 - low_3027) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_3027[low_3027] > arr_3027[mid_3027]) {
            swap_3027(arr_3027, low_3027, mid_3027);
        }
        if (arr_3027[low_3027] > arr_3027[high_3027]) {
            swap_3027(arr_3027, low_3027, high_3027);
        }
        if (arr_3027[mid_3027] > arr_3027[high_3027]) {
            swap_3027(arr_3027, mid_3027, high_3027);
        }
        swap_3027(arr_3027, mid_3027, high_3027);
    }

    static int partition_3027(int[] arr_3027, int low_3027, int
high_3027) {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
    }
}
```

```

        medianOfThree_3027(arr_3027, low_3027, high_3027);

        int pivot_3027 = arr_3027[high_3027]; // Sekarang arr[high]
sudah berisi nilai median
        int i_3027 = (low_3027 - 1);

        for (int j_3027 = low_3027; j_3027 <= high_3027 - 1; j_3027++)
        {
            // Jika elemen saat ini lebih kecil dari atau sama
dengan pivot
            if (arr_3027[j_3027] < pivot_3027) {
                // Increment indeks elemen yang lebih kecil
                i_3027++;
                swap_3027(arr_3027, i_3027, j_3027);
            }
        }
        swap_3027(arr_3027, i_3027 + 1, high_3027);
        return (i_3027 + 1);
    }

    static void quickSort_3027(int[] arr_3027, int low_3027, int
high_3027) {
        if (low_3027 < high_3027) {
            int pi_3027 = partition_3027(arr_3027, low_3027,
high_3027);

            quickSort_3027(arr_3027, low_3027, pi_3027 - 1);
            quickSort_3027(arr_3027, pi_3027 + 1, high_3027);
        }
    }

    public static void printArr_3027(int[] arr_3027) {
        for (int i_3027 = 0; i_3027 < arr_3027.length; i_3027++) {
            System.out.print(arr_3027[i_3027] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr_3027 = { 10, 7, 8, 9, 1, 5 };
        int N_3027 = arr_3027.length;
        System.out.print("Data Sebelum di Urutkan: ");
        printArr_3027(arr_3027);

        quickSort_3027(arr_3027, 0, N_3027 - 1);

        System.out.print("Data Terurut quicksort: ");
        printArr_3027(arr_3027);
    }
}

```

5. ShellSort_2511533027.java

- Program ini digunakan untuk memahami proses pengurutan data menggunakan algoritma Shell Sort.

- Program bekerja dengan membandingkan elemen yang memiliki jarak tertentu (gap) dan secara bertahap memperkecil jarak tersebut hingga menjadi satu.
- Method `shellSort_2511533027()` digunakan untuk melakukan proses pengurutan dengan teknik pergeseran elemen berdasarkan nilai gap.
- Nilai gap awal ditentukan dari setengah panjang array dan akan terus dibagi dua hingga mencapai nilai satu.
- Program menggunakan konsep insertion sort pada setiap kelompok elemen yang terbentuk berdasarkan gap.
- Pada method `main`, program mendeklarasikan array integer yang belum terurut dan menjalankan proses Shell Sort.
- Hasil akhir program berupa array yang telah terurut secara ascending menggunakan algoritma Shell Sort.

```
package pekan8_2511533027;

public class ShellSort_2511533027 {
    public static void shellSort_2511533027(int[] A_3027) {
        int n_3027 = A_3027.length;
        int gap_3027 = n_3027 / 2;
        while (gap_3027 > 0) {
            for (int i_3027 = gap_3027; i_3027 < n_3027; i_3027++)
            {
                int temp_3027 = A_3027[i_3027];
                int j_3027 = i_3027;
                while (j_3027 >= gap_3027 && A_3027[j_3027 -
gap_3027] > temp_3027) {
                    A_3027[j_3027] = A_3027[j_3027 -
gap_3027];
                    j_3027 = j_3027 - gap_3027;
                }
                A_3027[j_3027] = temp_3027;
            }
            gap_3027 = gap_3027 / 2;
        }
    }

    public static void main(String[] args) {
        int[] data_3027 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

        System.out.print("Sebelum: ");
        printArray_3027(data_3027);

        shellSort_2511533027(data_3027);

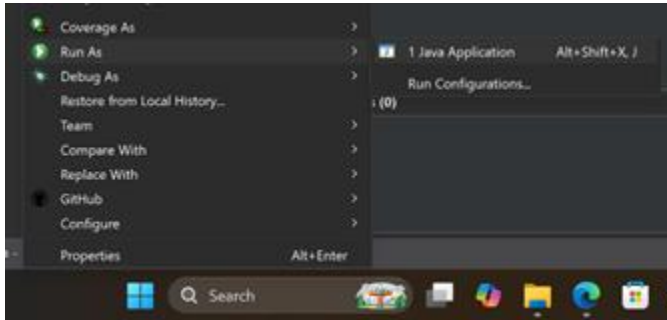
        System.out.print("Sesudah (Shell Sort): ");
        printArray_3027(data_3027);
    }
}
```

```

public static void printArray_3027(int[] arr_3027) {
    for (int i_3027 : arr_3027) System.out.print(i_3027 + " ");
    System.out.println();
}
}

```

5. Kompilasi dan Menjalankan Program



2.2 Analisis Hasil dan Pembahasan

1. BubbleSortGUI_2511533027.java

- Hasil: Program berhasil melakukan proses pengurutan data menggunakan algoritma Bubble Sort secara bertahap melalui tampilan GUI. Data yang dimasukkan pengguna dapat diurutkan secara menaik setelah seluruh langkah sorting selesai dijalankan. Program juga berhasil menampilkan proses perbandingan dan pertukaran elemen dalam bentuk visual serta mencatat setiap langkah pada area log.
- Pembahasan: Pada program ini, algoritma Bubble Sort bekerja dengan membandingkan dua elemen yang bersebelahan. Jika elemen pertama lebih besar daripada elemen kedua, maka kedua elemen akan ditukar. Proses tersebut dilakukan secara bertahap setiap kali tombol "Langkah Selanjutnya" ditekan. Elemen yang sedang dibandingkan ditandai dengan warna tertentu sehingga pengguna dapat melihat proses sorting secara lebih jelas. Dari hasil percobaan terlihat bahwa nilai terbesar secara bertahap berpindah ke posisi paling akhir hingga seluruh data tersusun secara menaik.

2. MergeSort_2511533027.java

- Hasil: Program berhasil mengurutkan data menggunakan algoritma Merge Sort. Array yang awalnya berisi data acak dapat diurutkan secara menaik setelah proses sorting selesai dilakukan. Program juga berhasil menampilkan data sebelum dan sesudah pengurutan melalui output pada console.
- Pembahasan: Pada program ini, algoritma Merge Sort bekerja dengan membagi array menjadi beberapa bagian yang lebih kecil hingga setiap bagian hanya berisi

satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali sambil membandingkan elemen yang ada sehingga menghasilkan urutan yang benar. Proses pembagian dilakukan secara rekursif melalui method `sort()`, sedangkan proses penggabungan dilakukan oleh method `merge_3027()`. Dari hasil percobaan terlihat bahwa metode ini mampu mengurutkan data dengan baik meskipun jumlah data cukup banyak.

3. MergeSortGUI_2511533027.java

- a) Hasil: Program berhasil menampilkan proses pengurutan data menggunakan algoritma Merge Sort secara visual melalui GUI. Pengguna dapat melihat setiap tahap penggabungan data hingga seluruh array menjadi terurut. Program juga menampilkan catatan langkah-langkah yang terjadi selama proses sorting berlangsung.
- b) Pembahasan: Pada program ini, konsep Merge Sort divisualisasikan menggunakan tampilan GUI sehingga proses pengurutan lebih mudah dipahami. Program terlebih dahulu membagi array menjadi beberapa bagian kecil, kemudian menggabungkannya kembali secara bertahap. Saat proses merge berlangsung, elemen yang sedang dibandingkan diberi warna berbeda dan hasil penggabungan ditampilkan langsung pada array utama. Dari hasil percobaan dapat dilihat bahwa proses pembagian dan penggabungan berlangsung secara sistematis hingga menghasilkan urutan data yang benar.

4. QuickSort_2511533027.java

- a) Hasil: Program berhasil melakukan pengurutan data menggunakan algoritma Quick Sort. Data array yang awalnya belum terurut dapat tersusun secara menaik setelah proses sorting selesai dijalankan. Program juga berhasil menampilkan data sebelum dan sesudah pengurutan melalui output console.
- b) Pembahasan: Pada program ini, algoritma Quick Sort menggunakan teknik pembagian data berdasarkan nilai pivot. Pivot dipilih menggunakan metode Median of Three untuk memperoleh hasil yang lebih optimal. Setelah pivot ditentukan, elemen yang lebih kecil dari pivot ditempatkan di sebelah kiri, sedangkan elemen yang lebih besar ditempatkan di sebelah kanan. Proses ini dilakukan secara rekursif hingga seluruh bagian array terurut. Dari hasil percobaan terlihat bahwa Quick Sort mampu mengurutkan data dengan cepat karena membagi masalah menjadi beberapa bagian yang lebih kecil.

5. ShellSort_2511533027.java

- a) Hasil: Program berhasil mengurutkan data menggunakan algoritma Shell Sort. Array yang awalnya berisi data acak dapat diurutkan secara menaik setelah proses sorting selesai dilakukan. Program juga berhasil menampilkan data sebelum dan sesudah pengurutan pada console.
- b) Pembahasan: Pada program ini, algoritma Shell Sort bekerja dengan menggunakan nilai gap atau jarak tertentu untuk membandingkan elemen-elemen dalam array. Nilai gap awal ditentukan dari setengah panjang array dan akan terus diperkecil hingga bernilai satu. Pada setiap tahap, elemen-elemen yang memiliki jarak sesuai gap dibandingkan dan dipindahkan ke posisi yang tepat. Ketika gap bernilai satu, proses pengurutan menjadi mirip dengan Insertion Sort namun lebih cepat karena sebagian data sudah mendekati posisi yang benar. Dari hasil percobaan terlihat bahwa Shell Sort mampu mengurangi jumlah pergeseran data sehingga proses pengurutan menjadi lebih efisien.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *sorting* berfungsi untuk menyusun data agar lebih rapi dan mudah dianalisis. Dalam praktikum ini, kami telah mempraktikkan empat jenis algoritma, yaitu Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort. Selain itu, kami juga menerapkan visualisasi GUI pada Bubble Sort dan Merge Sort untuk melihat proses pengurutannya secara langsung.

Dari hasil uji coba, seluruh program yang dibuat berhasil mengurutkan data acak menjadi urutan menaik (*ascending*). Setiap algoritma memiliki karakteristiknya sendiri: Bubble Sort bekerja dengan membandingkan elemen yang bersebelahan; Merge Sort membagi dan menggabungkan data; Quick Sort memanfaatkan nilai *pivot* sebagai acuan pembagian; sedangkan Shell Sort mengandalkan nilai *gap* untuk mempercepat prosesnya.

Melalui praktikum ini, kami tidak hanya memahami konsep dan perbedaan cara kerja dari masing-masing algoritma, tetapi juga mampu mengimplementasikannya langsung menggunakan bahasa pemrograman Java. Kehadiran GUI pada beberapa modul juga sangat membantu dalam memvisualisasikan bagaimana data tersebut bergerak hingga terurut.

DAFTAR PUSTAKA

- [1] Wahyudi, *Materi Struktur Data: Sorting*. Padang: PPT Perkuliahan, 2026.
- [2] P. Deitel dan H. Deitel, *Java How to Program*, 12th ed. Hoboken, NJ, USA: Pearson Education, 2020.

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“SORTING”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHARAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas rahmat dan karunia-Nya sehingga laporan tugas praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan tugas ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Sorting menggunakan studi kasus Playlist Lagu (mengurutkan data lagu secara ascending). Pada praktikum ini, penulis membuat program Java yang dapat mengelola data lagu dan juga durasinya dengan Algoritma Sorting (Merge Sort, Shell Sort, atau Quick Sort).

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
1.4 Instruksi Tugas	2
BAB II	4
2.1 Kelas Lagu_2511533027	4
2.2 Kelas Sorting_2511533027	4
BAB III.....	10
3.1 Kesimpulan	10
DAFTAR PUSTAKA	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam aktivitas sehari-hari, data yang menumpuk perlu diorganisasi dengan baik agar proses pencarian dan pengelolaannya berjalan optimal. Salah satu pendekatan yang umum digunakan adalah teknik pengurutan (*sorting*) data. Melalui *sorting*, penyajian informasi menjadi lebih rapi dan mempercepat waktu pencarian.

Studi kasus pada tugas ini menerapkan pengurutan pada data *playlist* lagu yang berisi atribut judul, penyanyi, dan durasi. Kriteria pengurutan yang diambil adalah berdasarkan durasi lagu dari yang terkecil hingga terbesar (*ascending*) dengan menggunakan metode Quick Sort.

Mekanisme Quick Sort mengandalkan penentuan nilai *pivot* untuk memecah kumpulan data. Data yang bernilai di bawah *pivot* akan dikelompokkan ke sebelah kiri, sedangkan data yang lebih besar akan dipindahkan ke sebelah kanan. Proses pembagian ini dilakukan secara terus-menerus hingga susunan data menjadi valid. Implementasi menggunakan bahasa pemrograman Java dalam tugas ini diharapkan dapat memberikan pemahaman praktis mengenai cara kerja logis dari algoritma Quick Sort.

1.2 Tujuan

Adapun tujuan dari tugas ini adalah:

1. Memahami konsep dasar pengurutan data (*sorting*) dalam struktur data.
2. Mempelajari cara kerja algoritma Quick Sort.
3. Mengimplementasikan algoritma Quick Sort menggunakan bahasa pemrograman Java.
4. Mengurutkan data playlist lagu berdasarkan durasi dari yang terpendek hingga yang terpanjang.
5. Melatih kemampuan dalam membuat program yang menggunakan array dan algoritma sorting.

1.3 Manfaat

Manfaat yang diperoleh adalah:

1. Menambah pemahaman mengenai konsep sorting pada struktur data.
2. Mengetahui cara kerja algoritma Quick Sort dalam mengurutkan data.

3. Melatih kemampuan pemrograman Java, khususnya dalam penggunaan array dan method.
4. Membantu memahami penerapan algoritma sorting pada kasus nyata, seperti pengelolaan playlist lagu.
5. Menjadi bekal untuk mempelajari algoritma dan struktur data yang lebih kompleks pada perkuliahan selanjutnya.

1.4 Instruksi Tugas

1. Aturan Penamaan
 1. Nama Kelas: gunakan NIM lengkap. Contoh: Sorting_2411532014
 2. Nama variabel & method: wajib pakai 4 digit terakhir NIM. Contoh: dataLagu_2014, shellSort_2014()
 3. Jika memilih Quick Sort, method utama harus quickSort_xxxx(), jika Merge: mergeSort_xxxx(), jika Shell: shellSort_xxxx()

2. Struktur Program

Kelas Lagu (sederhana)

Atribut: judul (String), penyanyi (String), durasi (int)

Kelas Sorting_NIM

1. Minimal harus ada:
 2. Array dataLagu_xxxx untuk menyimpan maksimal 20 lagu
 3. Method inputData_xxxx() – untuk mengisi data awal (minimal 7 lagu)
 4. Method pilihAlgoritma – cukup implementasikan SATU dari tiga di bawah:
 - a. shellSort_xxxx() – urutkan berdasarkan judul A-Z
 - b. quickSort_xxxx() – urutkan berdasarkan durasi ascending
 - c. mergeSort_xxxx() – urutkan berdasarkan judul A-Z
 5. Method tampilData_xxxx() – menampilkan sebelum dan sesudah sorting
3. Contoh Output
- === Sorting Playlist NIM: 2411532014 === Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:

1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik

3. La Perla - 196 detik

Data Setelah Quick Sort (Durasi Asc):

1. La Perla - 196 detik

2. La Rumba Del Perdon - 252 detik

3. Mio Cristo Piange Diamanti - 270 detik

BAB II PEMBAHASAN

2.1 Kelas Lagu_2511533027

Program Lagu_2511533027.java digunakan sebagai class untuk menyimpan data lagu yang akan diproses pada program sorting. Class ini memiliki tiga atribut yaitu judul_3027, penyanyi_3027, dan durasi_3027 yang berfungsi untuk menyimpan informasi judul lagu, nama penyanyi, dan lama durasi lagu dalam satuan detik. Selain itu, class ini juga memiliki constructor yang digunakan untuk menginisialisasi nilai atribut saat objek lagu dibuat sehingga setiap data lagu dapat disimpan dan dikelola dengan lebih terstruktur.

```
package pekan8_2511533027;

public class Lagu_2511533027 {
    String judul_3027;
    String penyanyi_3027;
    int durasi_3027;

    public Lagu_2511533027(String judul_3027, String penyanyi_3027, int
durasi_3027) {
        this.judul_3027 = judul_3027;
        this.penyanyi_3027 = penyanyi_3027;
        this.durasi_3027 = durasi_3027;
    }
}
```

2.2 Kelas Sorting_2511533027

Program Sorting_2511533027.java merupakan program utama yang digunakan untuk mengelola data playlist lagu sekaligus mengimplementasikan algoritma Shell Sort, Quick Sort, dan Merge Sort. Program dimulai dengan meminta pengguna memasukkan jumlah lagu beserta informasi judul, penyanyi, dan durasi lagu yang kemudian disimpan ke dalam array objek. Selanjutnya program menampilkan menu pilihan algoritma sorting yang dapat dipilih oleh pengguna. Shell Sort digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis, Quick Sort digunakan untuk mengurutkan data berdasarkan durasi lagu secara ascending, sedangkan Merge Sort digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis dengan teknik pembagian dan penggabungan data secara rekursif. Setelah proses pengurutan selesai, program menampilkan data sebelum dan sesudah sorting sehingga pengguna dapat melihat perubahan urutan data dengan jelas. Program juga memberikan pilihan kepada pengguna

untuk mencoba algoritma sorting lain tanpa harus memasukkan ulang data lagu yang telah dibuat sebelumnya.

```
package pekan8_2511533027;

import java.util.Scanner;

public class Sorting_2511533027 {
    Lagu_2511533027[] dataLagu_3027 = new Lagu_2511533027[20];
    int jumlah_3027 = 0;

    public void inputData_3027() {
        Scanner input_3027 = new Scanner(System.in);
        System.out.print("Masukkan jumlah lagu (minimal 7): ");
        jumlah_3027 = input_3027.nextInt();
        input_3027.nextLine();
        if (jumlah_3027 < 7) {
            System.out.println("Jumlah lagu minimal 7!");
            System.exit(0);
        }
        for (int i_3027 = 0; i_3027 < jumlah_3027; i_3027++) {
            System.out.println("\nData Lagu ke-" + (i_3027 + 1));
            System.out.print("Judul Lagu      : ");
            String judul_3027 = input_3027.nextLine();
            System.out.print("Penyanyi       : ");
            String penyanyi_3027 = input_3027.nextLine();
            System.out.print("Durasi (detik): ");
            int durasi_3027 = input_3027.nextInt();
            input_3027.nextLine();
            dataLagu_3027[i_3027] = new Lagu_2511533027(judul_3027,
            penyanyi_3027, durasi_3027);
        }
    }

    public void tampilData_3027() {
        for (int i_3027 = 0; i_3027 < jumlah_3027; i_3027++) {
            System.out.println((i_3027 + 1) + ". " +
dataLagu_3027[i_3027].judul_3027 + " - " + dataLagu_3027[i_3027].durasi_3027 +
" detik");
        }
    }

    public void swap_3027(int i_3027, int j_3027) {
        Lagu_2511533027 temp_3027 = dataLagu_3027[i_3027];
        dataLagu_3027[i_3027] = dataLagu_3027[j_3027];
        dataLagu_3027[j_3027] = temp_3027;
    }

    // ===== SHELL SORT =====

    public void shellSort_3027() {
        int gap_3027 = jumlah_3027 / 2;
        while (gap_3027 > 0) {
            for (int i_3027 = gap_3027; i_3027 < jumlah_3027; i_3027++) {
                Lagu_2511533027 temp_3027 = dataLagu_3027[i_3027];
                int j_3027 = i_3027;
                while (j_3027 >= gap_3027 &&
```

```

        dataLagu_3027[j_3027 -
gap_3027].judul_3027.compareToIgnoreCase(temp_3027.judul_3027) > 0) {
            dataLagu_3027[j_3027] =
                dataLagu_3027[j_3027 - gap_3027];
            j_3027 -= gap_3027;
        }
        dataLagu_3027[j_3027] = temp_3027;
    }
    gap_3027 /= 2;
}

// ===== QUICK SORT =====

public int partition_3027(int low_3027, int high_3027) {
    int pivot_3027 = dataLagu_3027[high_3027].durasi_3027;
    int i_3027 = low_3027 - 1;
    for (int j_3027 = low_3027; j_3027 < high_3027; j_3027++) {
        if (dataLagu_3027[j_3027].durasi_3027 <= pivot_3027) {
            i_3027++;
            swap_3027(i_3027, j_3027);
        }
    }
    swap_3027(i_3027 + 1, high_3027);
    return i_3027 + 1;
}

public void quickSort_3027(int low_3027, int high_3027) {
    if (low_3027 < high_3027) {
        int pi_3027 =
            partition_3027(low_3027, high_3027);
        quickSort_3027(low_3027, pi_3027 - 1);
        quickSort_3027(pi_3027 + 1, high_3027);
    }
}

// ===== MERGE SORT =====

public void merge_3027(int left_3027, int mid_3027, int right_3027) {
    int n1_3027 = mid_3027 - left_3027 + 1;
    int n2_3027 = right_3027 - mid_3027;
    Lagu_2511533027[] L_3027 =
        new Lagu_2511533027[n1_3027];
    Lagu_2511533027[] R_3027 = new Lagu_2511533027[n2_3027];
    for (int i_3027 = 0; i_3027 < n1_3027; i_3027++) {
        L_3027[i_3027] = dataLagu_3027[left_3027 + i_3027];
    }
    for (int j_3027 = 0; j_3027 < n2_3027; j_3027++) {
        R_3027[j_3027] = dataLagu_3027[mid_3027 + 1 + j_3027];
    }
    int i_3027 = 0;
    int j_3027 = 0;
    int k_3027 = left_3027;
    while (i_3027 < n1_3027 &&
        j_3027 < n2_3027) {
        if (L_3027[i_3027].judul_3027.compareToIgnoreCase(
R_3027[j_3027].judul_3027) <= 0) {
            dataLagu_3027[k_3027] = L_3027[i_3027];
            i_3027++;

```

```

        } else {
            dataLagu_3027[k_3027] = R_3027[j_3027];
            j_3027++;
        }
        k_3027++;
    }
    while (i_3027 < n1_3027) {
        dataLagu_3027[k_3027++] = L_3027[i_3027++];
    }
    while (j_3027 < n2_3027) {
        dataLagu_3027[k_3027++] = R_3027[j_3027++];
    }
}

public void mergeSort_3027(int left_3027, int right_3027) {
    if (left_3027 < right_3027) {
        int mid_3027 = (left_3027 + right_3027) / 2;
        mergeSort_3027(left_3027, mid_3027);
        mergeSort_3027(mid_3027 + 1, right_3027);
        merge_3027(left_3027,
                    mid_3027,
                    right_3027);
    }
}

// ===== MAIN =====

public static void main(String[] args) {
    Scanner input_3027 = new Scanner(System.in);
    Sorting_2511533027 playlist_3027 = new Sorting_2511533027();
    playlist_3027.inputData_3027();
    boolean ulang_3027 = true;
    while (ulang_3027) {
        System.out.println("\n=====");
        System.out.println("Pilih Algoritma Sorting");
        System.out.println("1. Shell Sort");
        System.out.println("2. Quick Sort");
        System.out.println("3. Merge Sort");
        System.out.print("Pilihan: ");
        int pilihan_3027 = input_3027.nextInt();
        System.out.println("\nData Sebelum Sorting:");
        playlist_3027.tampilData_3027();
        switch (pilihan_3027) {
            case 1:
                playlist_3027.shellSort_3027();
                System.out.println("\nData Setelah Shell Sort (Judul A-
Z):");
                playlist_3027.tampilData_3027();
                break;
            case 2:
                playlist_3027.quickSort_3027(0, playlist_3027.jumlah_3027
- 1);
                System.out.println("\nData Setelah Quick Sort (Durasi
Asc):");
                playlist_3027.tampilData_3027();
                break;
            case 3:
                playlist_3027.mergeSort_3027(0, playlist_3027.jumlah_3027
- 1);

```

```

        System.out.println("\nData Setelah Merge Sort (Judul A-
Z):");
        playlist_3027.tampilData_3027();
        break;
        default:
            System.out.println("Pilihan tidak valid!");
        }
        System.out.println("\nIngin mencoba algoritma lain?");
        System.out.println("1. Ya");
        System.out.println("2. Tidak");
        System.out.print("Pilihan: ");
        int jawab_3027 = input_3027.nextInt();
        if (jawab_3027 != 1) {
            ulang_3027 = false;
        }
    }
    System.out.println("\nProgram selesai.");
}
}
}

```

Output:

```

Terminated: Sorting 2511533027 [Java Application] C:\Users\hpl.p2\pool\plugins\org.eclipse.jdt.launcher.hotspot\jre.full.win32.x86_64.21.0.8.v20250724-1412\re\bin\javaw.exe (Jun 5, 2026, 10:29:36 PM - 10:31:59 PM elapsed: 0:02:22.984) [pid: 3468]
Masukkan jumlah lagu (minimal 7): 7

Data Lagu ke-1
Judul Lagu : when i was your man
Penyanyi : bruno mars
Durasi (detik): 456

Data Lagu ke-2
Judul Lagu : nobody gets me
Penyanyi : sza
Durasi (detik): 230

Data Lagu ke-3
Judul Lagu : pelangi
Penyanyi : hivi
Durasi (detik): 550

Data Lagu ke-4
Judul Lagu : remaja
Penyanyi : hivi
Durasi (detik): 443

Data Lagu ke-5
Judul Lagu : flatline
Penyanyi : justin bieber
Durasi (detik): 444

Data Lagu ke-6
Judul Lagu : sewindu
Penyanyi : tulus
Durasi (detik): 230

Data Lagu ke-7
Judul Lagu : that should be me
Penyanyi : justin bieber
Durasi (detik): 418

=====

```

```

C:\Users\hp\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Jun 5, 2026, 10:29:36 PM - 10:31:59 PM elapsed: 0:02:22.984) [pid: 3488]
=====
Pilih Algoritma Sorting
1. Shell Sort
2. Quick Sort
3. Merge Sort
Pilihan: 2

Data Sebelum Sorting:
1. when i was your man - 456 detik
2. nobody gets me - 230 detik
3. pelangi - 550 detik
4. remaja - 443 detik
5. flatline - 444 detik
6. sewindu - 230 detik
7. that should be me - 418 detik

Data Setelah Quick Sort (Durasi Asc):
1. nobody gets me - 230 detik
2. sewindu - 230 detik
3. that should be me - 418 detik
4. remaja - 443 detik
5. flatline - 444 detik
6. when i was your man - 456 detik
7. pelangi - 550 detik

Ingin mencoba algoritma lain?
1. Ya
2. Tidak
Pilihan: 1

=====
Pilih Algoritma Sorting
1. Shell Sort
2. Quick Sort
3. Merge Sort
Pilihan: 3

```

```

C:\Users\hp\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Jun 5, 2026, 10:29:36 PM - 10:31:59 PM elapsed: 0:02:22.984) [pid: 3488]
Data Sebelum Sorting:
1. nobody gets me - 230 detik
2. sewindu - 230 detik
3. that should be me - 418 detik
4. remaja - 443 detik
5. flatline - 444 detik
6. when i was your man - 456 detik
7. pelangi - 550 detik

Data Setelah Merge Sort (Judul A-Z):
1. flatline - 444 detik
2. nobody gets me - 230 detik
3. pelangi - 550 detik
4. remaja - 443 detik
5. sewindu - 230 detik
6. that should be me - 418 detik
7. when i was your man - 456 detik

Ingin mencoba algoritma lain?
1. Ya
2. Tidak
Pilihan: 1

=====
Pilih Algoritma Sorting
1. Shell Sort
2. Quick Sort
3. Merge Sort
Pilihan: 1

```

```

Data Sebelum Sorting:
1. flatline - 444 detik
2. nobody gets me - 230 detik
3. pelangi - 550 detik
4. remaja - 443 detik
5. sewindu - 230 detik
6. that should be me - 418 detik
7. when i was your man - 456 detik

Data Setelah Shell Sort (Judul A-Z):
1. flatline - 444 detik
2. nobody gets me - 230 detik
3. pelangi - 550 detik
4. remaja - 443 detik
5. sewindu - 230 detik
6. that should be me - 418 detik
7. when i was your man - 456 detik

Ingin mencoba algoritma lain?
1. Ya
2. Tidak
Pilihan: 2

Program selesai.

```

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian program, dapat disimpulkan bahwa algoritma **Shell Sort**, **Quick Sort**, dan **Merge Sort** dapat digunakan dengan baik untuk melakukan pengurutan data playlist lagu. Setiap algoritma berhasil mengurutkan data sesuai dengan kriteria yang telah ditentukan, yaitu berdasarkan judul lagu maupun durasi lagu. Program juga mampu menerima input data dari pengguna, menampilkan data sebelum proses sorting, serta menampilkan hasil setelah proses pengurutan dilakukan. Melalui tugas ini, pemahaman mengenai konsep algoritma sorting dan penerapannya dalam bahasa pemrograman Java menjadi lebih baik, terutama dalam mengolah dan mengurutkan data secara efisien.

DAFTAR PUSTAKA

- [1] Wahyudi, *Materi Struktur Data: Sorting*. Padang: PPT Perkuliahan, 2026.
- [2] P. Deitel dan H. Deitel, *Java How to Program*, 12th ed. Hoboken, NJ, USA: Pearson Education, 2020.